

Practical-6

Aim: Implementation of Chain Matrix Multiplication using dynamic programming.

Algorithm:

Chain Matrix Multiplication

Algorithm:

Input: No. of matrices, dimensions of matrices.
Output: Minimum no. of scalar multiplication.

Step 1: start the program
Step 2: Read the number of scalar matrices n
Step 3: Read the dimensions of the matrices using the array P[]
Step 4: create a DP table m[i][j], where m[i][j] stores the minimum multiplication cost for matrices i through j.
Step 5: Initialize the cost of multiplying one matrix as 0.
Step 6: Consider different chain length from 2 to n.
Step 7: For each chain try every possible position k to split the chain.
Step 8: calculate the multiplication cost :
$$m[i][k] + m[k+1][j] + P[i-1] * P[k] * P[j]$$

Step 9: solve the minimum cost in m[i][j]
Step 10: Display the minimum number of scalar multiplication
Step 11: Stop.

Program:

```
#include <iostream>
```

```
using namespace std;
```

```
void matrixChainOrder(int p[], int n)
```

```
{
```

Name: Tarmisha

EnrollmentNo: 92460118029

```
int m[20][20];

// Cost is zero when multiplying one matrix
for (int i = 1; i < n; i++)
    m[i][i] = 0;

// L is chain length
for (int L = 2; L < n; L++)
{
    for (int i = 1; i < n - L + 1; i++)
    {
        int j = i + L - 1;
        m[i][j] = 999999;

        for (int k = i; k < j; k++)
        {
            int cost = m[i][k] +
                        m[k + 1][j] +
                        p[i - 1] * p[k] * p[j];

            if (cost < m[i][j])
                m[i][j] = cost;
        }
    }
}
```

```
cout << "Minimum number of scalar multiplications = "  
    << m[1][n - 1] << endl;  
}  
  
int main()  
{  
    // Dimensions of matrices:  
    // A1 = 10x20  
    // A2 = 20x30  
    // A3 = 30x40  
    // A4 = 40x30  
  
    int p[] = {10, 20, 30, 40, 30};  
  
    int n = sizeof(p) / sizeof(p[0]);  
  
    matrixChainOrder(p, n);  
  
    return 0;  
}
```

Output:

```
Minimum number of scalar multiplications = 30000  
  
=== Code Execution Successful ===
```

Time complexity:

Time complexity:

```
for (d=1; d<n; d++) → O(n)
{
  for (i=1; i<=n-d; i++) { → O(n)
    for (k=i; k<=j-1; k++) { → O(n)
      }
    }
  }
```

$T = O(n^2)$

Best case: $O(n^3)$ - All three loops execute.
Average case: $O(n^3)$ - All possible split positions checked.
Worst case: $O(n^3)$ - All three nested loop execute fully.

Conclusion: Chain Matrix Multiplication was successfully implemented using Dynamic Programming. It finds the best order to multiply matrices with the minimum number of calculations. This method saves time by avoiding repeated calculations and provides an efficient solution to the problem.